DEPLOYMENT STEPS & AUTOMATION

- Create remote database instance, connect & test
- Create cloud server instance and connect
- Dockerize your backend project
- Connect your repo to your server & pull Project
- Run app
- Write CI/CD build & auto Deployment script
- Bundle your frontend and deploy

STEP 1 CREATE RDS CLOUD DATABASE

1. Create a rds database instance(use at least a 2gig ram instance type i.e t2.small).
2. Check the security group to be sure of the port.
    a. If the db port is not exposed , edit the inbound rule to expose it .
3. Test & connect to the database from the workbench and from the project codebase.

STEP 1 CREATE EC2 CLOUD INSTANCE

1. Create ec2 instance(use at least 2gig ram t3.micro, t3.small)
2. Configure security group.
    a. Edit inbound rule and register/expose the port of your service.
3. Connect to the server via ssh.
    a. ssh -i "bank-server-instance.pem" ec2-user@ec2-3-141-18-52.us-east-2.compute.amazonaws.com
4. Install docker on your server
    a. sudo dnf update -y ⇒ update the system
    b. sudo dnf install docker -y ⇒install docker
    c. sudo systemctl start docker ⇒ start docker service
    d. sudo systemctl enable docker ⇒ enable docker to start on boot
    e. sudo usermod -a -G docker ec2-user ⇒ Add ec2-user to docker group (so you can run docker without sudo)
    f. newgrp docker ⇒ Refresh group membership (or log out and back in)
    g. sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose ⇒ install docker compose
    h. sudo chmod +x /usr/local/bin/docker-compose ⇒ granny permission to it
    i. docker --version ⇒ Verify Docker installation
    j. docker-compose --version ⇒ check docker compose version
    k. sudo systemctl status docker ⇒ check docker status
    l. docker ps ⇒ list running containers
    m. docker run hello-world ⇒Test Docker with a simple container

STEP 3: SETUP SSH TO BE ABLE TO CLONE GITBUB REPO PROJECTS FROM YOUR SERVER

1. ssh-keygen -t ed25519 -C "your_email@gmail.com" ⇒generate a new SSH key on your server
2. cat ~/.ssh/id_ed25519.pub ⇒ copy the output
3. Login your github account and go to settings ⇒ SSH & GPS key. Create a new SSH and paste it there.
4. ssh -T git@github.com ⇒ Test the ssh connection

STEP 4: CLONE YOUR PROJECT REPO, BUILD DOCKER IMAGE AND RUN

1. mkdir projects ⇒ make a directory
2. sudo dnf install git -y ⇒ install git
3. git --version ⇒ check git version
4. git clone git@github.com:phegondev/phegon-bank.git ⇒ clone the project
5. nano .env ⇒ Create an .env file and past you env variables there
6. docker-compose up –build ⇒ build the image and run the container without detaching
7. `docker-compose up --build -d` ⇒ rebuild the image and runs the container in background(detach mode)
8. `docker-compose up -d` ⇒ build and run the image only if it's not already running
9. docker builder prune ⇒ clear docker build cash to free some space if your server capacity is small
10. free -h ⇒ check server ram
11. df -h ⇒ check server space
12. docker logs "container_id or container name" ⇒ view the logs
13. docker logs d976d607e0cb ⇒ view the logs
14. docker-compose stop ⇒ stop the container only
15. docker-compose down ⇒stop and remove the container

16. docker images ⇒ to list all images on your server
17. docker ps -a ⇒ to list all running and non running container
18. docker ps ⇒ list only running containers

STEP 5: IMPLEMENT CI/CD TO AUTO DEPLOY WHEN CODE IS MERGED TO MASTER BRANCH

1. .github/workflows/deploy.yml ⇒ create this file in the master branch
2. .github/workflows/validate.yml ⇒ and this too to validate when code is pushed
3. Now write the validation script (already in the github link)
4. And also the deployment script (already in the github link)

STEP 6: GENERATE AN SSH KEY PAIR FOR GITHUB ACTIONS(this will be used by github to auto access your server for deployment)

1. ssh-keygen -t rsa -b 4096 -C "github-actions-deploy" -f ~/.ssh/github_actions_rsa ⇒ execute this on your server
2. cat ~/.ssh/github_actions_rsa.pub >> ~/.ssh/authorized_keys ⇒Still on your EC2 server, copy the public key to your authorized_keys file. This allows connections from any client that has the corresponding private key
3. cat ~/.ssh/authorized_keys ⇒ to see the content. Just to verify the public key was copied and saved there
4. cat ~/.ssh/github_actions_rsa ⇒ this will display the private key. Copy it to be stored in github.
5. Go to the project repo in github → Settings → Secrets and variables → Actions
6. Click New repository secret. Give it a name as SSH_PRIVATE_KEY same variable you have in your deploy.yml file . also add SSH_HOST the value will be your public ip.
7. Now. when copying the private key copy everything (including the -----BEGIN and -----END lines)